

1. โครงสร้าง Hash Table ชนิด Open Addressing กำหนดวิธีการแก้การชนกันเป็นแบบ Double Hashing กำหนด TableSize ให้เป็น 19% ของการทำ Rehashing เมื่อ Table ใกล้จะเต็มที่มีมากกว่า 75% ¹⁴
- 1.1 ให้นศ.ทำการ insert ข้อมูลจำนวน 8 ตัว ดังต่อไปนี้ เข้าไปใน HashTable ดังกล่าว โดยใช้วิธีการแก้การชนกันแบบ Double Hashing กำหนด $f(i) = 1 * R - (X \text{ MOD } R)$ 323, 17, 969, 868, 340, 370, 357, 391 รวม insert ทุกตัวเสร็จสิ้น ² ³ ⁴ ³ **10 คะแนน**

พื้นที่สำหรับการคิดแต่ละตัว โดยใช้สูตร และฟังก์ชัน ไม่ต้องแสดงขั้นตอนของโปรแกรม

$$\begin{aligned}
 h_i(x) &= (\text{hash}(x) + F(i)) \% \text{TableSize} \\
 h_0(323) &= ((323 \% 19) + 0) \% 19 = 0 \\
 h_0(17) &= ((17 \% 19) + 0) \% 19 = 17 \\
 h_0(969) &= ((969 \% 19) + 0) \% 19 = 0 \\
 F(i) &= 1 * (19 - (969 \% 19)) = 19 \\
 h_1(969) &= ((0 + 19) \% 19) = 19 \\
 F(i) &= 2 * (19 - (969 \% 19)) = 38 \\
 h_2(969) &= ((0 + 38) \% 19) = 19 \\
 h_0(868) &= ((868 \% 19) + 0) \% 19 = 13 \\
 h_0(340) &= ((340 \% 19) + 0) \% 19 = 17 \\
 F(i) &= 1 * (19 - (340 \% 19)) = 17 \\
 h_1(340) &= ((17 + 17) \% 19) = 15 \\
 F(i) &= 2 * (19 - (340 \% 19)) = 34 \\
 h_2(340) &= ((17 + 34) \% 19) = 17 \\
 h_0(370) &= ((370 \% 19) + 0) \% 19 = 9 \\
 h_0(357) &= ((357 \% 19) + 0) \% 19 = 15 \\
 F(i) &= 1 * (19 - (357 \% 19)) = 17 \\
 h_1(357) &= ((15 + 17) \% 19) = 13 \\
 F(i) &= 2 * (19 - (357 \% 19)) = 34 \\
 h_2(357) &= ((15 + 34) \% 19) = 17 \\
 F(i) &= 3 * (19 - (357 \% 19)) = 51 \\
 h_3(357) &= ((15 + 51) \% 19) = 5 \\
 h_0(391) &= ((391 \% 19) + 0) \% 19 = 17 \\
 F(i) &= 1 * (19 - (391 \% 19)) = 17 \\
 h_1(391) &= ((17 + 17) \% 19) = 15 \\
 F(i) &= 2 * (19 - (391 \% 19)) = 34 \\
 h_2(391) &= ((17 + 34) \% 19) = 17 \\
 F(i) &= 3 * (19 - (391 \% 19)) = 51 \\
 h_3(391) &= ((17 + 51) \% 19) = 17
 \end{aligned}$$

ตอบ นำตัวเลขมาใส่ในตาราง Hash Table นี้ ช่วงใดเป็น Active ให้เขียน Info เป็น A ด้วย

	Element	Info
0 1	323	A
1 2		E
2 3		E
3 4		E
4 5		E
5 6	370	A
6 7		E
7 8	357	A
8 9		E
9 10	370	A
10 11		E
11 12	340	A
12 13		E
13 14	868	A
14 15		E
15 16	969	A
16 17		E
17 18	17	A
18 19		E

1. โครงสร้าง Hash Table ชนิด Open Addressing กำหนดวิธีการแก้การชนกันเป็นแบบ Double Hashing กำหนด TableSize ให้เป็น 19% ของการทำ Rehashing เมื่อ Table เกล็ดจะเต็มที่มีมากกว่า 75% ¹⁴
- 1.1 ให้นศ.ทำการ insert ข้อมูลจำนวน 8 ตัว ดังต่อไปนี้ เข้าไปใน HashTable ดังกล่าว โดยใช้วิธีการแก้การชนกันแบบ Double Hashing กำหนด $f(i) = I * R - (X \text{ MOD } R)$ 323,17,969,868,340,370,357,391 รวม insert ทุกตัวเสร็จสิ้น ² ³ ⁴ ³ **10 คะแนน**

พื้นที่สำหรับการคิดแต่ละตัว โดยใช้สูตร และฟังก์ชัน ไม่ต้องแสดงขั้นตอนของโปรแกรม

$$\begin{aligned}
 h_i(x) &= (\text{hash}(x) + F(i)) \% \text{TableSize} \\
 h_1(323) &= (323 \% 19) + 17 = 15 & F(1) &= 1 * (17 - (323 \% 19)) = 68 \\
 h_0(323) &= ((323 \% 19) + 0) \% 19 = 0 & F(i) &= 2 * (17 - (340 \% 19)) = 34 & h_4(357) &= ((357 \% 19) + 68) \% 19 = 7 \\
 h_0(17) &= ((17 \% 19) + 0) \% 19 = 17 & h_2(340) &= ((340 \% 19) + 34) \% 19 = 13 & h_0(370) &= ((370 \% 19) + 0) \% 19 = 11 \\
 h_0(969) &= ((969 \% 19) + 0) \% 19 = 0 & F(i) &= 3 * (17 - (340 \% 19)) = 51 & F(i) &= 1 * (17 - (370 \% 19)) = 17 \\
 F(i) &= 1 * (17 - (969 \% 19)) = 17 & h_3(340) &= ((340 \% 19) + 51) \% 19 = 11 & h_1(17) &= ((17 \% 19) + 17) \% 19 = 0 \\
 h_1(17) &= ((0 + 17) \% 19) = 17 & h_0(370) &= ((370 \% 19) + 0) \% 19 = 9 & F(i) &= 2 * (17 - (370 \% 19)) = 34 \\
 F(i) &= 2 * (17 - (969 \% 19)) = 34 & h_2(357) &= ((357 \% 19) + 0) \% 19 = 15 & h_2(370) &= ((370 \% 19) + 34) \% 19 = 9 \\
 h_2(17) &= ((0 + 34) \% 19) = 15 & F(i) &= 3 * (17 - (357 \% 19)) = 51 & F(i) &= 3 * (17 - (370 \% 19)) = 51 \\
 h_0(868) &= ((868 \% 19) + 0) \% 19 = 13 & h_1(357) &= ((357 \% 19) + 17) \% 19 = 13 & h_3(370) &= ((370 \% 19) + 51) \% 19 = 5 \\
 h_0(340) &= ((340 \% 19) + 0) \% 19 = 17 & F(i) &= 2 * (17 - (357 \% 19)) = 34 & & \\
 F(i) &= 1 * (17 - (340 \% 19)) = 17 & h_2(357) &= ((357 \% 19) + 34) \% 19 = 17 & & \\
 & & F(i) &= 3 * (17 - (357 \% 19)) = 51 & & \\
 & & h_3(357) &= ((357 \% 19) + 51) \% 19 = 9 & &
 \end{aligned}$$

ตอบ นำตัวเลขมาใส่ในตาราง Hash Table นี้ ช่วงใดเป็น Active ให้เขียน Info เป็น A ด้วย

	Element	Info
0 1	323	A
1 2		E
2 3		E
3 4		E
4 5		E
5 6	370	A
6 7		E
7 8	357	A
8 9		E
9 10	370	A
10 11		E
11 12	340	A
12 13		E
13 14	868	A
14 15		E
15 16	969	A
16 17		E
17 18	17	A
18 19		E

1.2 หลังจากใส่ข้อมูลลงใน Hash Table ดังกล่าว ครบทุกตัว ถามว่า เกิดการชนกันทั้งสิ้น กี่ครั้ง

ตอบ 12 ครั้ง

1.3 หลังจากใส่ข้อมูลลงใน Hash Table ดังกล่าว ครบทุกตัว ต้องใส่ข้อมูลเพิ่มอย่างน้อยกี่ตัวถึงจะมีการ Rehashing

ตอบ 6 ตัว $8 * 100/19 = 42.10\%$ ไม่ถึง 70%
 $8+6=14 \rightarrow 14 * 100/19 = 73.68\%$

2. จงใส่ข้อมูลตัวเลขต่อไปนี้ 64, 19, 40, 39, 21, 30, 25, 29, 24, 90 ลงใน Binary Heap ทางซ้ายมือ ให้เขียนเฉพาะภาพผลลัพธ์สุดท้าย insert เสร็จแล้วเท่านั้น 4 คะแนน

2.1 ให้ทำการ Delete Min 2 ครั้ง แล้วเขียนผลลัพธ์แต่ละครั้งลงใน Array ช่องว่างทางขวามือ

2.2 หลังจากนั้น ให้ทำการ Insert ข้อมูลตัวเลข 12 เข้าไปใน Binary Heap

ให้เขียนผลลัพธ์ข้อมูลในนี้ ให้เขียนผลลัพธ์ของการ Delete Min แต่ละครั้ง

0	
1	19
2	21
3	25
4	24
5	39
6	40
7	30
8	64
9	29
10	90
11	

21
24
25
29
39
40
30
64
90

24
29
25
64
39
40
30
90

12
24
25
29
39
40
36
90
64

3. กำหนดข้อมูลเป็น input สำหรับการทำให้ insertion sort จำนวน 10 ตัว ดังนี้

60, 9, 44, 18, 31, 22, 38, 14, 53, 1

3.1 จงเขียนผลของการทำงาน หลังจากทำงานรอบที่ 4 และรอบที่ 8 (After P = 4 และ After P = 8) เสร็จแล้วทำตาม Function ในบทที่ 7 พร้อมทั้งเขียน Position Move ให้ถูกต้อง

		Position Move
เริ่มต้น		
After P = 4		
After P = 8		

3.2 จงเขียนขั้นตอนการทำงานของ Function Insertion Sort ขณะที่ทำงานในรอบที่ 5 (จากข้อที่ 3.1) ให้เขียนค่าในตัวแปรต่างๆ มีค่าเป็นเท่าไร โดยละเอียด

ทำงาน Function Insertion Sort

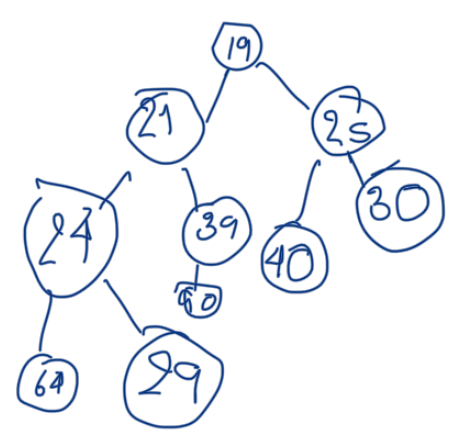
รอบที่ 5 P = 5

Tmp =

2

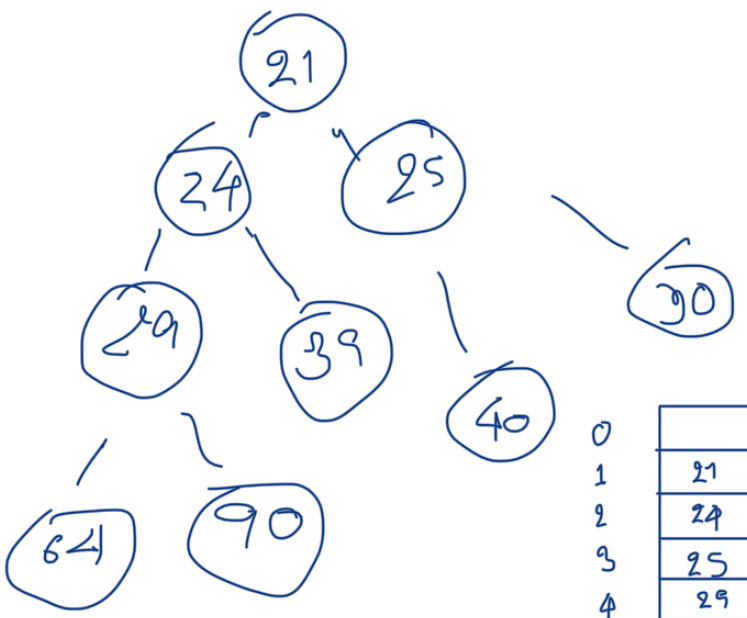
0		0		0		0	
1	64	1	19	1	19	1	19
2		2	64	2	39	2	24
3		3	40	3	40	3	40
4		4		4	64	4	64
5		5		5		5	39
6		6		6		6	
7		7		7		7	
8		8		8		8	
9		9		9		9	
10		10		10		10	
11		11		11		11	

64, 19, 40, 39, 21, 30, 25, 29, 24, 90



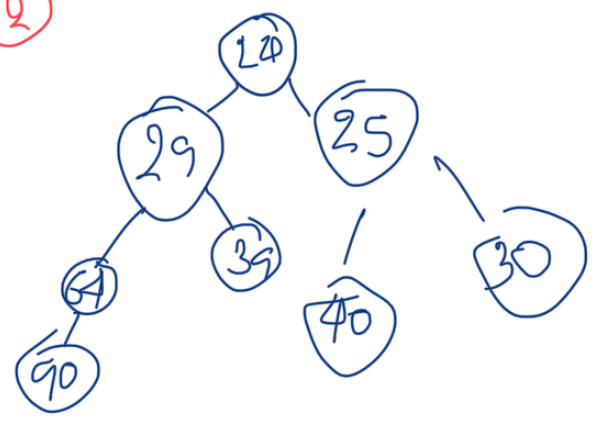
0		0		0		0	
1	19	1	19	1	19	1	19
2	24	2	21	2	21	2	21
3	30	3	25	3	25	3	25
4	64	4	64	4	29	4	24
5	39	5	39	5	39	5	39
6	40	6	40	6	40	6	40
7		7	30	7	30	7	30
8		8		8	64	8	64
9		9		9		9	29
10		10		10		10	90
11		11		11		11	

delete Min ①



0		0	
1	21	1	24
2	24	2	29
3	25	3	25
4	29	4	64
5	39	5	39
6	40	6	40
7	30	7	30
8	64	8	90
9	90	9	
10		10	
11		11	

②



1

0	
1	19
2	21
3	25
4	24
5	39
6	40
7	30
8	64
9	29
10	90
11	

hole = 1

tmp = 31

- ① hole = 1, $1^*2 \leq 13$; child = hole*2
 $2 \leq 13$; เปลี่ยนที่ของ 25 < 21 ที่จ
- ② hole = 2, $2^*2 \leq 13$; child = 4
 $4 \leq 13$; เปลี่ยนที่ของ 39 < 24 ที่จ
- ③ hole = 4, $4^*2 \leq 13$; child = 8
 $8 \leq 13$; เปลี่ยนที่ของ 29 < 64 และ child++
- ④ hole = 9, $9^*2 \leq 10$ ที่จ
 $18 \leq 10$ นกตจาก for loop

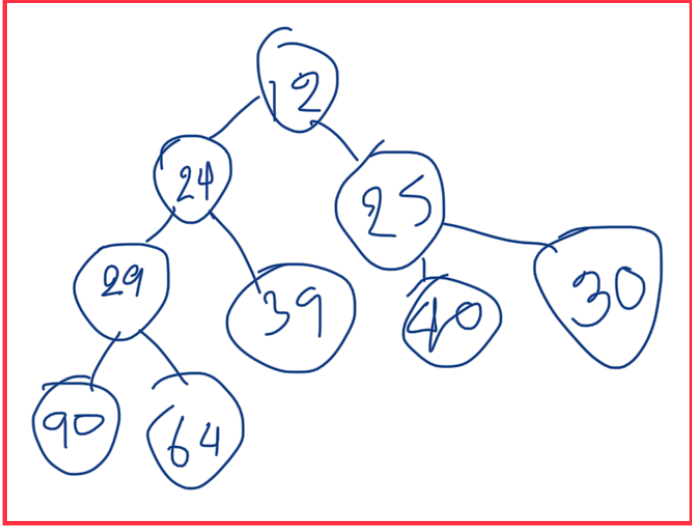
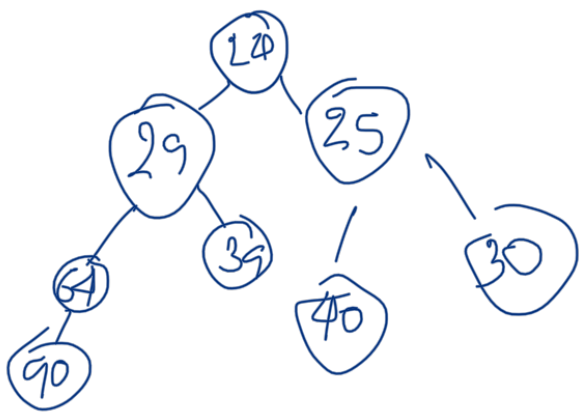
2

0	
1	21
2	24
3	25
4	29
5	39
6	40
7	30
8	64
9	90
10	
11	

hole = 2
tmp = 90

- ① hole = 1, $2 \leq 13$; child = 2
เปลี่ยนที่ของ 25 < 24 ที่จ
- ② hole = 2, $4 \leq 13$; child = 4
เปลี่ยนที่ของ 39 < 25 ที่จ
- ③ hole = 4, $8 \leq 13$; child = 8
เปลี่ยนที่ของ 90 < 64 ที่จ
- ④ hole = 8, $16 \leq 13$ ที่จ นกต loop
child 4i++

insert 12



0	
1	12
2	24
3	25
4	29
5	39
6	40
7	30
8	90
9	64
10	
11	

1.2 หลังจากใส่ข้อมูลลงใน Hash Table ดังกล่าว ครบทุกตัว ถามว่า เกิดการชนกันทั้งสิ้น กี่ครั้ง

ตอบ 12 ครั้ง

1.3 หลังจากใส่ข้อมูลลงใน Hash Table ดังกล่าว ครบทุกตัว ต้องใส่ข้อมูลเพิ่มอย่างน้อยกี่ตัวถึงจะมีการ Rehashing

ตอบ 6 ตัว $8 * 100/19 = 42.10\%$ ไม่ถึง 70%
 $8+6=14 \rightarrow 14 * 100/19 = 73.68\%$

2. จงใส่ข้อมูลตัวเลขต่อไปนี้ 64, 19, 40, 39, 21, 30, 25, 29, 24, 90 ลงใน Binary Heap ทางซ้ายมือ ให้เขียนเฉพาะภาพผลลัพธ์สุดท้าย insert เสร็จแล้วเท่านั้น 4 คะแนน

2.1 ให้ทำการ Delete Min 2 ครั้ง แล้วเขียนผลลัพธ์แต่ละครั้งลงใน Array ช่องว่างทางขวามือ

2.2 หลังจากนั้น ให้ทำการ Insert ข้อมูลตัวเลข 12 เข้าไปใน Binary Heap

ให้เขียนผลลัพธ์ข้อมูลในนี้ ให้เขียนผลลัพธ์ของการ Delete Min แต่ละครั้ง

0	
1	19
2	21
3	25
4	24
5	39
6	40
7	30
8	64
9	29
10	90
11	

21
24
25
29
39
40
30
64
90

24
29
25
64
39
40
30
90

12
24
25
29
39
40
36
90
64

3. กำหนดข้อมูลเป็น input สำหรับการทำให้ insertion sort จำนวน 10 ตัว ดังนี้

60, 9, 44, 18, 31, 22, 38, 14, 53, 1

3.1 จงเขียนผลของการทำงาน หลังจากทำงานรอบที่ 4 และรอบที่ 8 (After P = 4 และ After P = 8) เสร็จแล้วทำตาม Function ในบทที่ 7 พร้อมทั้งเขียน Position Move ให้ถูกต้อง

		Position Move
เริ่มต้น		
After P = 4	9 14 31 44 30 22 38 14 53 1	2
After P = 8	9 14 18 22 31 38 44 53 60 1	1

3.2 จงเขียนขั้นตอนการทำงานของ Function Insertion Sort ขณะที่ทำงานในรอบที่ 5 (จากข้อที่ 3.1) ให้เขียนค่าในตัวแปรต่างๆ มีค่าเป็นเท่าไร โดยละเอียด

ทำงาน Function Insertion Sort

รอบที่ 5 P = 5

Tmp =

③ 60, 9, 44, 18, 31, 22, 38, 14, 53, 1

After p = 1	9	60	44	18	31	22	38	14	53	1	P. moved
After p = 2	9	44	60	18	31	22	38	14	53	1	1
After p = 3	9	18	44	60	31	22	38	14	53	1	2
After p = 4	9	18	31	44	60	22	38	14	53	1	2
After p = 5	9	18	22	31	44	60	38	14	53	1	3
After p = 6	9	18	22	31	38	44	60	14	53	1	2
After p = 7	9	14	18	22	31	38	44	60	53	1	6
After p = 8	9	14	18	22	31	38	44	53	60	1	1
After p = 9	1	9	14	18	22	31	38	44	53	60	9

Function insertion sort

60 9 44 18 31 22 38 14 53 1
[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

For $\forall p; p = 1; 1 < 9$ ရှိ

tmp = 9

j =
For $\forall j; j = p = 1; 1 > 0 \ \&\& \ 9 < a[0];$ ရှိ

$a[j+1] = a[j-1];$

$a[1] = 60$

For $\forall j; j = 0; 0 > 0$ မရှိ

After p = 1 9 60 44 18 31 22 38 14 53 1
[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

For $\forall p; p = 2; 2 < 9$ ရှိ

tmp = 44

j =
For $\forall j; j = p = 2; 2 > 0 \ \&\& \ 44 < a[1];$ ရှိ

$a[j+1] = a[j-1];$

$= a[1]$

$a[2] = 60$

After p = 2 9 44 60 18 31 22 38 14 53 1
[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

- 3.3 จาก insertion sort ในบทที่ 7 ต้องแก้ไขโปรแกรมอย่างไร จึงจะสามารถนับการเลื่อนตำแหน่งของข้อมูลได้ (Position Move ของแต่ละรอบ) ให้เขียน Function Insertion Sort ที่ได้แก้ไขแล้วทั้ง Function

Template <class Comparable>

Void insert sort (Vector <comparable> &a){

For (int p = 1 ; p < a.size() ; p++){

Comparable tmp = a[p]

Int j;

For (j = p ; j > 0 && a[j-1] > tmp ; j--){

a[j] = a[j-1];

a[j] = tmp;

}

}

}

Template <class Comparable>

Void insert sort (Vector <comparable> &a)

4. กำหนดข้อมูลเป็น input ให้กับ function quicksort จำนวน 15 ตัว ดังนี้

79,57,36,17,14,5,9,23,38,19,45,31,11,4,3

สมมติข้อมูลอยู่ใน Array a กำหนดตัวเลข cutoff เท่ากับ 3 เมื่อทำงาน Function quicksort ตามหนังสือเรียน

4.1 จงเขียนข้อมูลใน Array a หลังจากใช้ pivot ตัวที่ 2 [ทำ quicksort ครั้งที่ 2] ทำงานเสร็จแล้ว

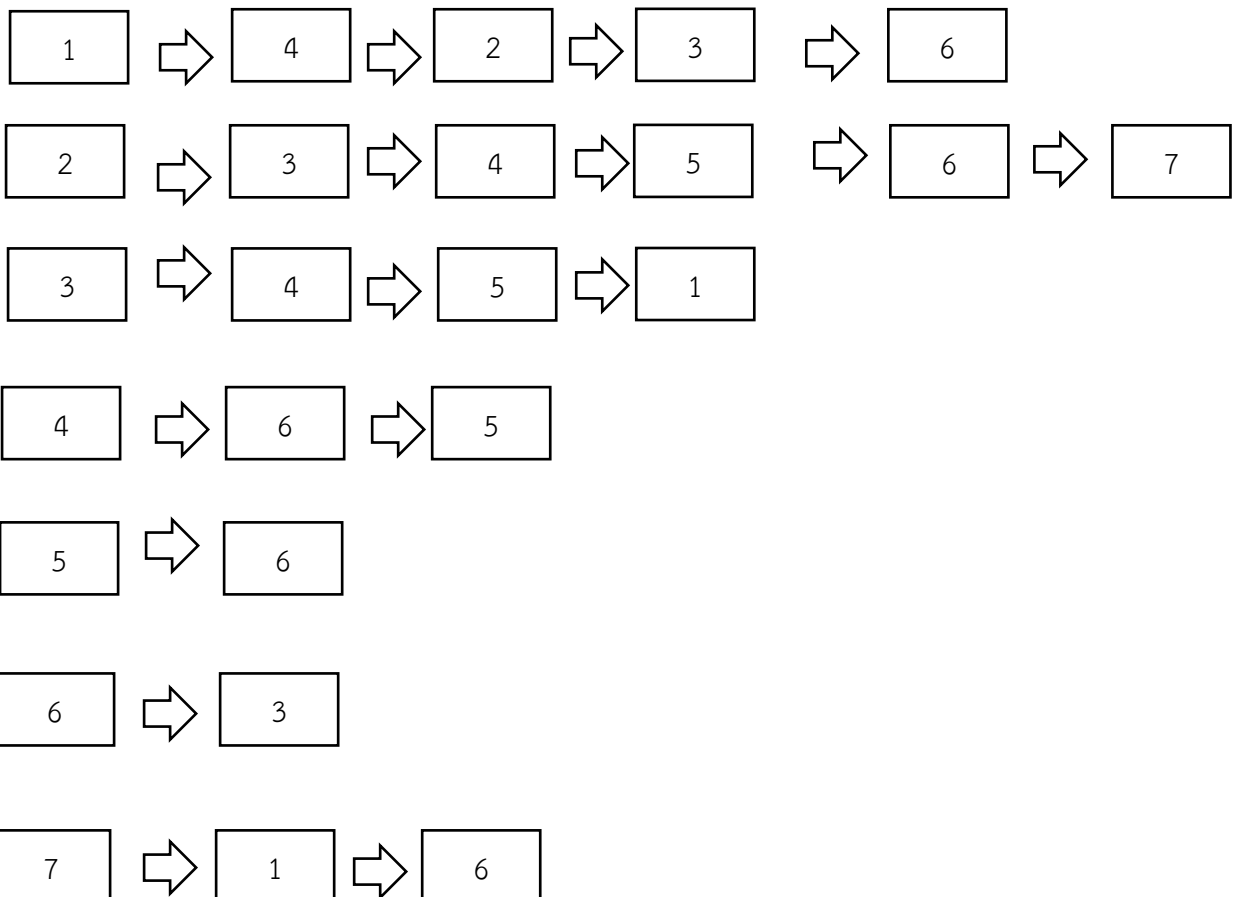
ตอบ Pivot ตัวที่ 2 มีค่าเท่ากับ

[9]	[8]	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]

4.2 จงเขียน pivot ที่ใช้ quicksort ทั้งหมดให้ครบทุกตัว ตามลำดับการเดิน pivot

ตอบ

5. กำหนดข้อมูลของกราฟหนึ่งเป็น Adjacency List มี 7 Vertex ดังภาพ



5.1 จงหา Short Path แบบ Unweighted โดยมี Vertex 4 เป็นจุดเริ่มต้น ต้องการให้ค้น.เขียนข้อมูล Array Tree ที่แสดง Know,Dv ทำทุก Vertex โดย Know เป็น T หรือ 1 ทั้งหมด

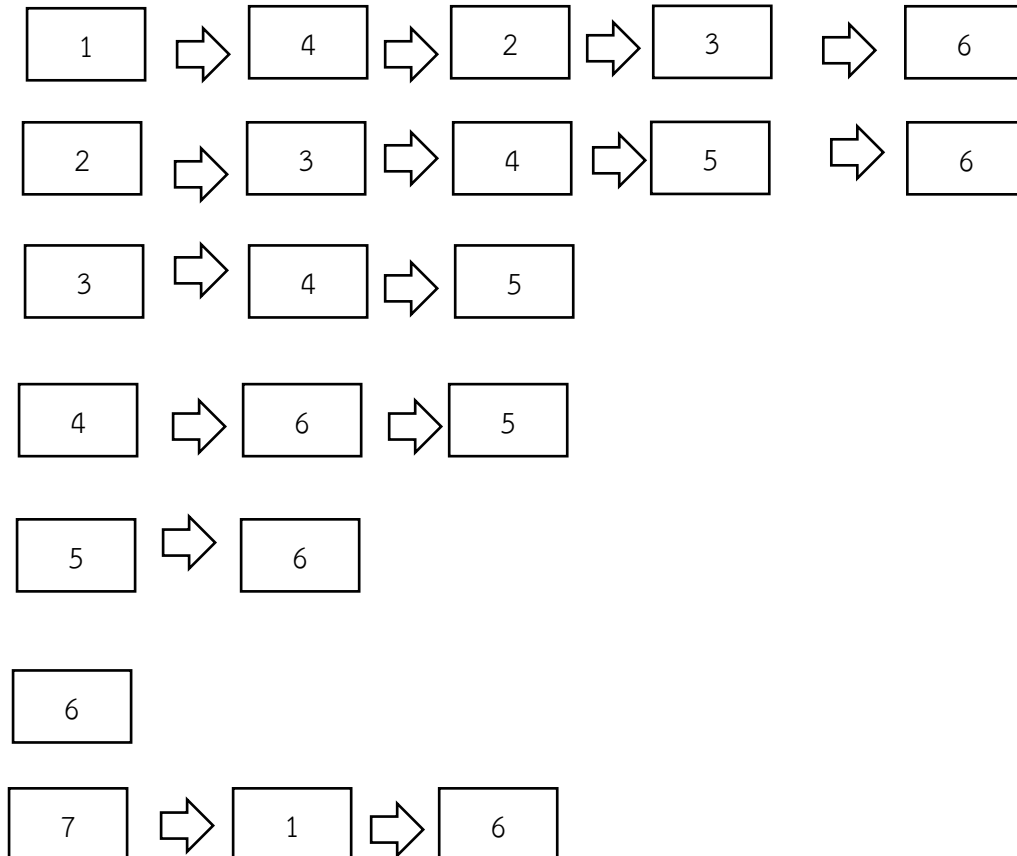
5.2 กำหนด Queue เพื่อใช้ประกอบ Algorithm ในการหา Shortest Path ขนาดของ Queue เป็น 7 ช่อง จงเขียนภาพของ Queue เพื่อแสดงข้อมูลใน Queue หลังจากทำ 5.1 เสร็จ โดยให้เขียนตำแหน่ง Font และ Back ด้วยว่าอยู่ที่ใด

	Know	Pivot (Dv)	Path(Pv)
1	T	3	3
2	T	4	1
3	T	2	6
4	T	0	0
5	T	1	4
6	T	1	4
7	F	5	2

Enqueue	4	6	5	3	1	2	7
Dequeue	4	6	5	3	1	2	

การนำข้อมูลเข้าออกจาก Queue ยึดตาม Function Enqueue และ Dequeue ในหนังสือที่ใช้เรียน บทที่ 3 เป็นหลัก

6. กำหนดข้อมูลของกราฟ DAG อันหนึ่งเป็น Adjacency List มี 7 Vertex ดังภาพ



จงแสดงวิธีหา Topological Sort ต้องการให้ นศ. เขียนข้อมูลของ Array Indegree

5

$$v = 4$$

$$w = 6 \quad w.\text{dist} = v.\text{dist} + 1$$

$$1 = 0 + 1$$

$$w.\text{path} = v$$

$$= 4$$

$$q.\text{enqueue}(w)$$

$$w = 5 \quad w.\text{dist} = v.\text{dist} + 1$$

$$1 = 0 + 1$$

$$w.\text{path} = v$$

$$= 4$$

$$q.\text{enqueue}(w)$$

$$v = 6$$

$$w = 3 \quad w.\text{dist} = v.\text{dist} + 1$$

$$2 = 1 + 1$$

$$w.\text{path} = v$$

$$= 6$$

$$q.\text{enqueue}(w)$$

$$v = 5$$

$$w = 6$$

$$v = 3$$

$$w = 4$$

$$w = 5$$

$$w = 1 \quad w.\text{dist} = v.\text{dist} + 1$$

$$3 = 2 + 1$$

$$w.\text{path} = v$$

$$= 3$$

$$q.\text{enqueue}(w)$$

$$v = 1$$

$$w = 4$$

$$w = 2 \quad w.\text{dist} = v.\text{dist} + 1$$

$$4 = 3 + 1$$

$$w.\text{path} = v$$

$$= 1$$

$$q.\text{enqueue}(w)$$

$$w = 3$$

$$w = 6$$

$$v = 2$$

$$w = 7 \quad w.\text{dist} = v.\text{dist} + 1$$

$$5 = 4 + 1$$

$$w.\text{path} = v$$

$$q.\text{enqueue}(w)$$

5.1 จงหา Short Path แบบ Unweighted โดยมี Vertex 4 เป็นจุดเริ่มต้น ต้องการให้ค้น.เขียนข้อมูล Array Tree ที่แสดง Know,Dv ทำทุก Vertex โดย Know เป็น T หรือ 1 ทั้งหมด

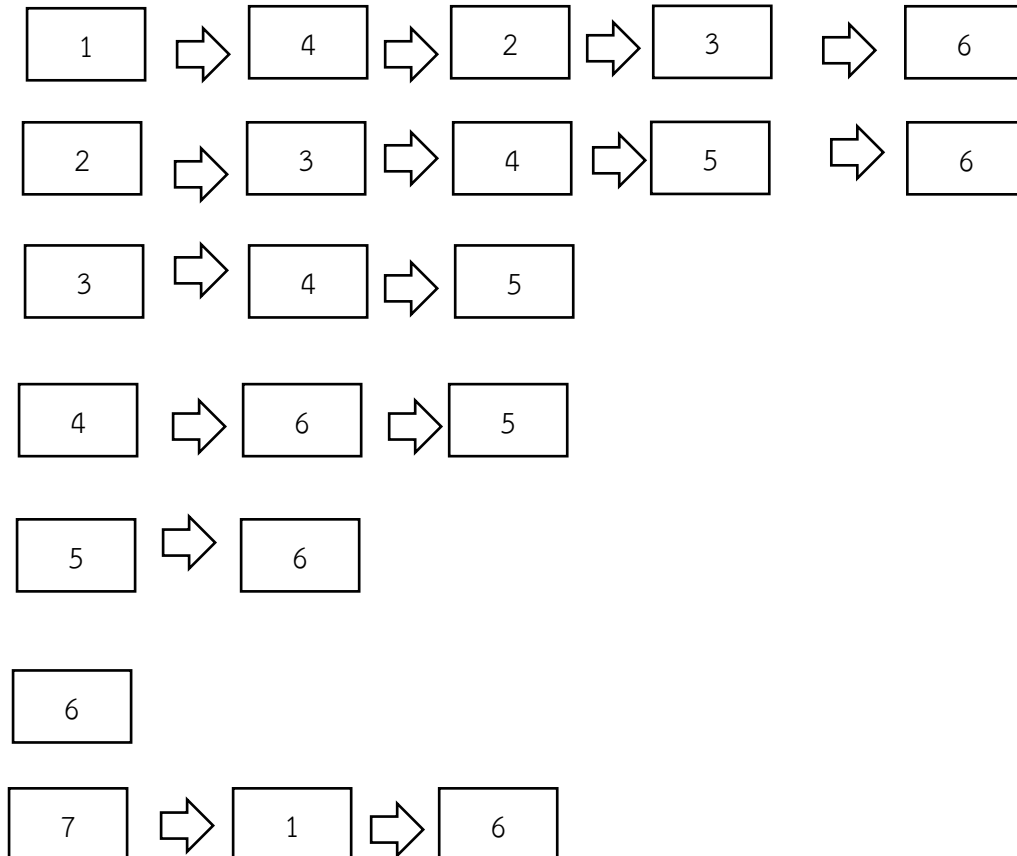
5.2 กำหนด Queue เพื่อใช้ประกอบ Algorithm ในการหา Shortest Path ขนาดของ Queue เป็น 7 ช่อง จงเขียนภาพของ Queue เพื่อแสดงข้อมูลใน Queue หลังจากทำ 5.1 เสร็จ โดยให้เขียนตำแหน่ง Front และ Back ด้วยว่าอยู่ที่ใด

	Know	Pivot (Dv)	Path(Pv)
1			
2			
3			
4			
5			
6			
7			

Enqueue							
Dequeue							

การนำข้อมูลเข้าออกจาก Queue ยึดตาม Function Enqueue และ Dequeue ในหนังสือที่ใช้เรียน บทที่ 3 เป็นหลัก

6. กำหนดข้อมูลของกราฟ DAG อันหนึ่งเป็น Adjacency List มี 7 Vertex ดังภาพ



จงแสดงวิธีหา Topological Sort ต้องการให้ ค้น. เขียนข้อมูลของ Array Indegree

- 6.2 กำหนด Queue เพื่อใช้ประกอบ Algorithm ในการหา Topological Sort ขนาดของ Queue เป็น 7 ช่อง จงเขียนภาพของ Queue เพื่อแสดงข้อมูลใน Queue หลังจากทำงานข้อ 6.1 เสร็จ โดยเขียนตำแหน่ง front และ back ด้วยว่าอยู่ที่ใด

V	เริ่มต้น	รอบที่ 1	รอบที่ 2	รอบที่ 3	รอบที่ 4	รอบที่ 5	รอบที่ 6	รอบที่ 7
1	1	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0
3	2	2	1	0	0	0	0	0
4	3	3	2	1	0	0	0	0
5	3	3	3	2	1	0	0	0
6	5	4	3	2	2	1	0	0
7	0	0	0	0	0	0	0	0

Queue	
6	6
5	5
4	4
3	3
2	2
1	1
0	7

Enqueue	7	1	2	3	4	5
Dequeue	7	1	2	3	4	5

- 6.3 เมื่อทำการ Topological Sort เสร็จแล้ว จงเขียน Topological Ordering ที่เป็นคำตอบของกราฟนี้

$v_7, v_1, v_2, v_3, v_4, v_5, v_6$

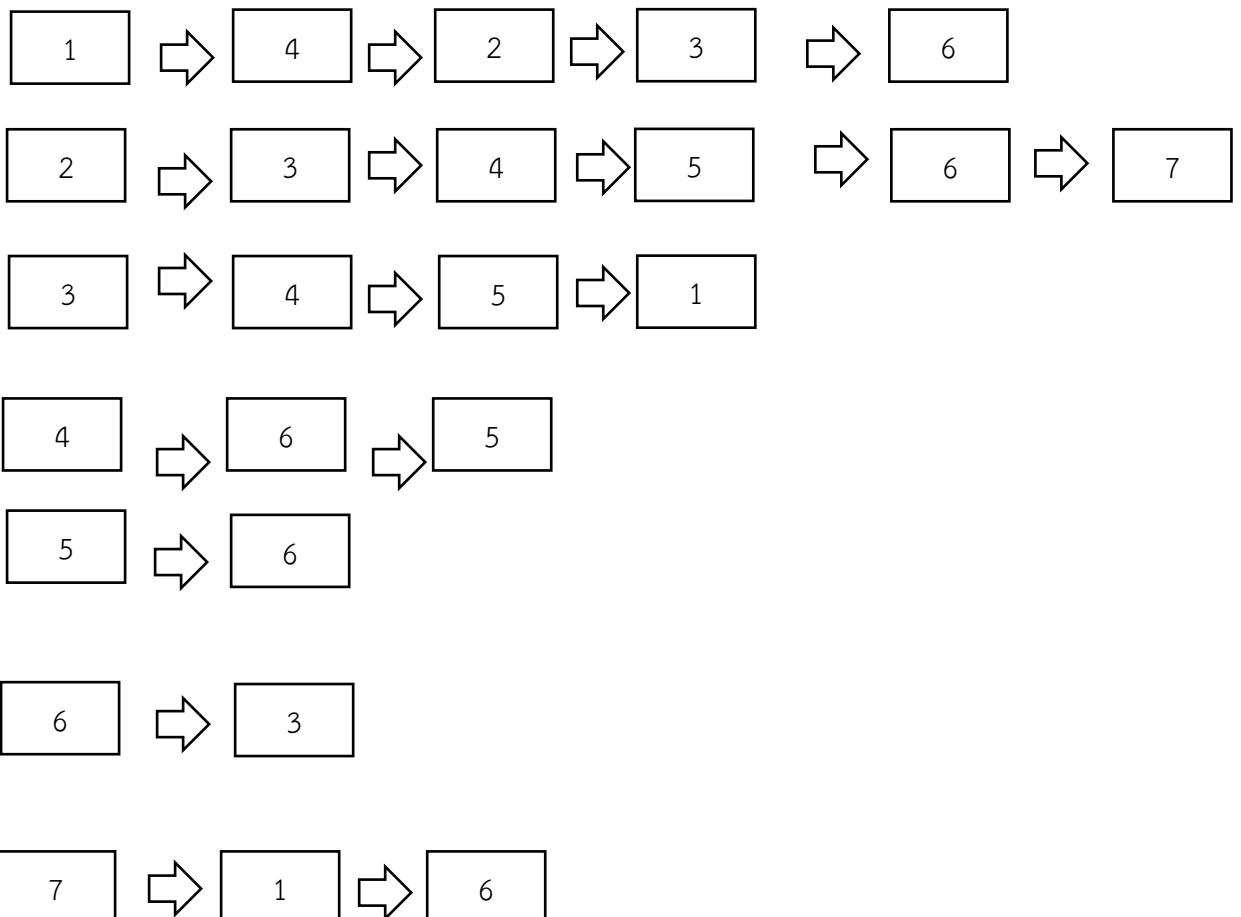
front = 0
back = 6

- 7.1 ในการทำ Quick Sort ทำไมสมาชิกใน Array ตำแหน่งที่อยู่ทางซ้ายและทางขวา จึงไม่ถูกนำมาเปรียบเทียบในขั้นตอนการทำ Partitioning

- 7.2 Complete Binary Tree ที่มีความสูงเท่ากับ 10 จะมีจำนวน Node อย่างน้อยเท่าไร และอย่างมากเท่าไร จะแสดงวิธีคำนวณด้วย

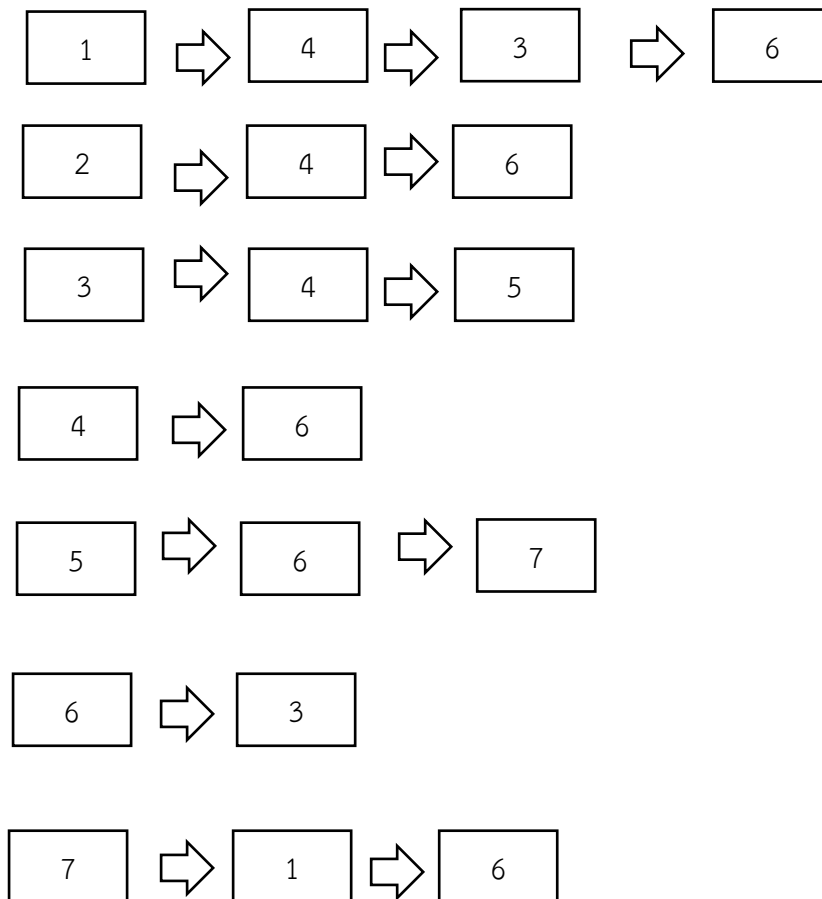
1. Double Hasing / Table = 17 / Rehashing = 70%
642,338,982,528,340,323,361,663
1.1 แสดงเฉพาะข้อมูลขณะใส่ 663
ถ้า Active ใส่ A
1.2 เมื่อใส่ข้อมูลครบเกิดการชนและแก้การชนทั้งหมดกี่ตัว
2. Heap 64,14,24,1,20,2,5,10,6,8,15,20
Binary Heap DeleteMin DeleteMin insert 4
3. จาก member function percolateDown() แก้โปรแกรมใช้นับจำนวนครั้งที่เลื่อนทั้งหมดได้
4. Insertion Sort
4.1 ข้อมูลทั้งหมด 15 ตัวเป็น Worst Case ลงในช่องเริ่มต้น(ใส่ผิดตั้งแต่แรกได้ 0 คะแนน)
หาผลการทำงานของ P แต่ละรอบ
After P = 5
After P = 10
4.2 insertion sort ทำ function นับรอบการเปรียบเทียบทั้งหมด -> เขียน function
4.3 จากข้อ 4.2 ข้อมูล 10,9,8,7,6,5,4,3,2,1 เกิดการเปรียบเทียบทั้งหมดกี่ครั้ง
5. Input Quicksort 15 ตัว
201,159,157,145,123,1001,93,81,76,64,42,33,21,12,5
Array a สมมติ cutoff = 3 เมื่อใช้ Quick Sort เรียงจากน้อยไปมาก
5.1 ในข้อมูล Array a หลังจากใช้งาน pivot ตัวที่ 3
5.2 จากข้อ 5.1
l = ? , j = ?
5.3 หลังทำ Quick Sort ครบ 15 ตัวมีการเรียก Insertion Sort ทั้งหมดกี่ครั้ง

6.



6.1 จงหา Short Path แบบ Unweighted โดยมี Vertex 1 เป็นจุดเริ่มต้น

7. การนำข้อมูลเข้าออกจาก Queue ยึดตาม Function Enqueue และ Dequeue ในหนังสือที่ใช้เรียน บทที่ 3 เป็นหลัก กำหนดข้อมูลของกราฟ DAG อันหนึ่งเป็น Adjacency List มี 7 Vertex ดังภาพ



7.1 เขียน Topological ของกราฟ

8.1 คุณสมบัติของ Hashing ในอุดมคติ 3 ข้อ

8.2 จากข้อ 8.1 ข้อใดเป็นไปได้ในทางปฏิบัติ สาเหตุ 2 ประการคือ

8.3 ในการทำ Quick Sort ตอนทำ Partitioning มีข้อมูลบางค่าไม่ได้นำไปทำ Partitioning ข้อมูลใดใน Array ที่ไม่ถูกนำไปทำ Partitioning เพราะอะไร

8.4 Complete Binary Tree สูง 15 มี Node สูงสุดและน้อยสุดเท่าไร

8.5 Complete Graph Vertex = 15 มี edge เท่าไร (ไม่ติดอยู่ในรูปสมการ)

8.6 เขียนขั้นตอนการหา Topological Sort ในกราฟตามคำสั่งที่เรียนมา